Technical Report — FastSpeech 2 Text-to-Speech for English and Telugu

Assignment: Implementation of FastSpeech 2-Based Text-to-Speech Synthesis
Datasets: Svarah (English, Indian accents) · IndicTTS Telugu
Reference: Ren et al., *FastSpeech 2: Fast and High-Quality End-to-End Text-to-Speech* (arXiv:2006.04558) · [ming024/FastSpeech2](https://github.com/ming024/FastSpeech2) (MIT)

> **Honesty statement.** Every number in this report is either (a) fixed by the code and testable, or (b) produced by an artefact in the repository (logs, plots, checkpoints, generated audio) and cited as such. Training results that have not yet been produced are marked `[PLACEHOLDER — pending training run]` and are **not** guessed. Alignment timing is labelled *measured* or *estimated* everywhere (see section 6).

1. Status summary

Component	State	Evidence
Data pipeline (download, subsample, audio decoding)	implemented + unit-tested	`tests/test_downloaders.py`, `tests/test_audio_helper.py`
CTC alignment engine (per-language)	implemented + unit-tested + smoke-tested	`tests/test_alignment_core.py`, `tools/smoke_test.py`
Feature extraction (mel, F0, energy, durations)	implemented + unit-tested	`tests/test_preprocessor_unit.py`
FastSpeech 2 model (shared) + variance controls	implemented + unit-tested	`tests/test_model_controls.py`
Training loop, checkpoints, resume	implemented + smoke-tested	`tests/test_inference_guard.py`, `tools/smoke_test.py`
Inference + variance plots + vocoder	implemented + smoke-tested	`tools/smoke_test.py`, `tests/test_vocoder.py`
Full training runs (5 h data, Colab GPU)	`[PLACEHOLDER — pending training run]`	section 10
Generated speech for the 10 test sentences	`[PLACEHOLDER — pending training run]`	section 10

The repository therefore ships a **complete, tested, end-to-end pipeline** whose small-scale end-to-end behaviour is proven by the smoke test, and it is ready to execute the real training runs on a Colab GPU.

2. Objective

Implement and evaluate FastSpeech 2 separately for English and Telugu: adapt the reference codebase (Telugu needs a new text front-end), train two

models on approximately 5 h training / 1 h validation data each, synthesise the assignment test sentences, and analyse the duration, pitch (F0) and energy predictions of the variance adaptor.

3. Architecture and modifications

3.1 What is kept from the reference

The feed-forward transformer encoder/decoder, the variance adaptor (duration / pitch / energy predictors with quantised embedding buckets), the loss functions, and the HiFi-GAN universal vocoder are taken from the MIT licensed reference with minimal edits. Model weights, folder organisation and attribution to `ming024/FastSpeech2` are preserved.

3.2 Structural changes

- one shared implementation (`fs2_core/`) with per-language entry points (`English/`, `Telugu/`), each with its own `config.yaml`, `checkpoints/` and `logs/` (assignment folder structure);
- phoneme durations come from **CTC forced alignment** instead of Montreal Forced Aligner TextGrids (MFA has no Telugu acoustic model; see section 6);
- mel / pitch / energy extraction is unchanged from the reference (pyworld `dio` + `stonemask` for F0, TacotronSTFT mel + energy).

3.3 English text front-end

Lower-cased word splitting, CMU-dict lookup with stress-preserving ARPAbet phones (`fs2_core/text/cmudict.py` + bundled `fs2_core/lexicon/librispeech-lexicon.txt`), `g2p_en` fallback for out-of-vocabulary words (needed for *androo/andaroo* in E4 and the split word *rhyth m* in E3). Number/time normalisation is handled by the standard `english_cleaners` chain.

3.4 Telugu text front-end (new)

`fs2_core/text/telugu.py` implements, from scratch:

- Unicode-aware cleaning (ZWNJ/ZWJ/avagraha handling, virama/halant rules);
- a **rule-based grapheme-to-phoneme** mapping (consonant + vowel-sign decomposition, inherent-vowel handling, anusvara place-of-articulation assimilation, visarga);
- number and time normalisation to Telugu words (cardinals, ordinals such as *12వ*, clock times such as *10:45*), because the test sentences mix digits and script;
- a weighted variant of the G2P that reports how many source characters each phoneme consumed, used to split word-level CTC timing (a documented heuristic, section 6).

4. Engineering audit and fixes (A–N)

The codebase was audited item by item and fixed as follows.

#	Area	Finding	Fix
A	Svarah downloader	read `item["audio"]` (wrong key), lost text/duration provenance	reads the `audio_filepath` audio column (falls back to `audio`), preserves `text` and reported `duration` verbatim in the manifest, records measured duration separately, counts every skipped row with a reason, and writes `download_report.json` with selected counts and hours
B	Svarah speaker IDs	fell back to one invented "svarah" speaker	speaker IDs come only from the official `meta_speaker_stats.csv` (joined on normalised text + duration, reporting mapped / unmatched / ambiguous counts and unique speakers). Without that file the pipeline documents a **single-voice fallback** (`multi_speaker: false`) and says so in the report, the manifest and the console. Invented per-utterance IDs are never generated
C	Audio decoding	only legacy `datasets` dicts handled	`data_prep/common.py:to_audio_dict` normalises modern `torchcodec` `AudioDecoder` objects (`get_all_samples()`), legacy dicts, `(array, sr)` tuples and bare arrays to `{"array": float32 mono, "sampling_rate": int}`; mixes multi-channel to mono; rejects empty/NaN/inf audio and invalid sampling rates with `AudioDecodeError`

#	Area	Finding	Fix
D	IndicTTS selection	decoded all 8 576 rows (6.4 GB) into memory	metadata-first two-pass design: durations are probed without retaining waveforms (dataset field, decoder metadata, or transient per-row decode), the 5 h/1 h selection is deterministic (fixed seed) and disjoint, and only the selected rows are decoded and written; the duration-filter audit (short/long/empty-text/decode-fail/gender) is recorded in `download_report.json`
E	Telugu speakers	`spk_<gender>` built ad hoc	the dataset's own `gender` field (ClassLabel) maps to exactly two stable speakers (`spk_female`, `spk_male`); rows with unusable gender labels are excluded and counted; the strategy is documented in the report JSON
F	Forced alignment	`torchaudio.functional.forced_align` (deprecated in torchaudio 2.8, removed in 2.9) + 22.05 kHz audio fed to a 16 kHz model + silent `<unk>` insertion	replaced with the maintained **`ctc-segmentation`** package (Kürzinger et al., LREC 2020) on top of the same wav2vec2 CTC models; audio is resampled to the feature extractor's sampling rate and normalised exactly as the extractor specifies; characters the model cannot spell are **rejected with an explicit error**; blank and word-delimiter token ids are verified against the tokenizer/model before aligning; torchaudio is no longer a dependency. No PyTorch downgrade is used or needed

#	Area	Finding	Fix
G	Alignment honesty	silent mock fallback on real failures; unvalidated durations	mock alignment exists only as an explicit `method: mock` for smoke tests; every real failure raises `AlignmentError`, is logged with the utterance id and reason in `align_report.json` and the utterance is excluded. Durations are guaranteed `>= 1` frame and to sum **exactly** to the mel-frame length (verified again by the preprocessor). Word boundaries are *measured* (CTC switch points); intra-word phoneme splits are *estimated* (English: phone-class weights; Telugu: character-weighted G2P shares) and labelled as such in per-utterance diagnostics
H	Manifest parsing	`line.split(",")` broke quoted fields; `header.index(0)` bug	`csv.DictReader` with required-column validation, duplicate-id detection, split-value validation and train/val disjointness checks; utterances are never re-split
I	Feature extraction	silent truncation when mel frames disagreed with durations; no NaN guards; invalid `stats.json` possible	mel/duration length equality is enforced (mismatch -> exclude + log); NaN/inf checks on waveform, mel, pitch, energy; zero/negative durations rejected; non-finite or degenerate (std = 0) statistics refuse to write `stats.json`; every exclusion is written to `preprocess_report.json`

#	Area	Finding	Fix
J	Variance control bug	`get_energy_embedding(..., p_control)` in both the phoneme- and frame-level branches: `e_control` was inert and `p_control` leaked into energy	both call sites pass `e_control`; regression tests assert exact, independent scaling of pitch/energy/duration controls (`tests/test_model_controls.py`)
K	Test sentences	drift risk between code and PDF	all ten sentences are stored verbatim in `fs2_core/test_sentences.py` (double spaces in E3/T3 and the attached *12ನ* in T4 preserved) and pinned by unit tests
L	Checkpoints / inference	inference silently used randomly initialised weights	checkpoints store model + optimizer + global step; `--resume` restores all three; **inference refuses to run without a checkpoint** (`tests/test_inference_guard.py`)
M	Vocoder	unchecked generator files	bundled HiFi-GAN zips, `config.json` consistency (22 050 Hz, upsample product = 256 = hop length), state-dict key check, finite-weight and finite-output checks around synthesis
N	Dependencies	environment drift (Colab Py3.13 / PyTorch 2.11 era)	`requirements.txt` targets current Colab (torch/torchaudio left to the runtime so CUDA is not broken; torchaudio no longer required at all), `tools/check_env.py` runs `pip check`, CUDA checks, import tests and prints a compatibility matrix

5. Dataset preparation

5.1 English — Svarah (gated, CC BY 4.0)

- 9.6 h, 117 speakers, 6 656 utterances, single published `test` split; schema: `audio_filepath` (audio), `duration`, `text`, demographics — no speaker id column.
- Selection: utterances with 1.0 s <= duration <= 15.0 s are eligible; a fixed seed (42) shuffles them and a greedy pass fills ~5 h train, then ~1 h val. The sets are disjoint by construction and verified.
- Outputs: `corpus/{speaker}/{id}.wav` (PCM-16, peak-normalised), `manifest.csv` (id, speaker, text, reported duration, measured duration, split, speaker source, gender) and `download_report.json` (selection counts/hours, skipped-row reasons, duration sources, speaker mapping report).
- Speaker mapping: see item B. The assignment's 117-speaker figures are dataset properties; whether individual utterances can be tied to those speakers depends on `meta_speaker_stats.csv` availability and the join statistics are printed honestly either way.

5.2 Telugu — IndicTTS Telugu (public)

- ~8 576 story utterances, fields `audio` / `text` / `gender`, one published `train` split; we create our own 5 h/1 h split (seed 42), disjoint and deterministic.
- Two stable speakers derived from the gender field (the corpus was recorded by one male and one female voice artist).
- Same outputs and report JSON as English; the duration-filter audit records how many rows were dropped and why.

6. Alignment methodology (what is measured and what is estimated)

Pipeline: wav -> resample to the ASR feature extractor's sampling rate (16 kHz) with its own normalisation -> wav2vec2 CTC posteriors (per language: `facebook/wav2vec2-large-960h-lv60-self` for English, `anuragshas/wav2vec2-large-xlsr-53-telugu` for Telugu) -> `ctc-segmentation` token-to-frame alignment -> word spans -> G2P phonemes with durations.

Measured (observed in the audio):

- when each transcript character sounds (CTC switch points);
- word boundaries and inter-word gaps, leading/trailing silence;
- the total speech coverage and per-token confidence scores (stored in the per-utterance `diagnostics` block).

Estimated (heuristic, labelled as such):

- how a word's measured span divides among its G2P phonemes. English uses phone-class duration weights (vowels longer than stops); Telugu uses the number of source characters each rule-based phoneme consumed. These are plausible priors, **not** observed intra-word timings.

Invariants enforced: every phoneme duration is an integer >= 1 frame and the durations sum exactly to the mel-frame length of the utterance (`n_samples // hop + 1`, verified against the actual STFT). Impossible cases (utterance too short for its phones, unsupported characters, empty audio) raise `AlignmentError`, are recorded in `align_report.json` with the

utterance id and excluded.

Mock alignment (uniform durations) exists only for the smoke test and is requested explicitly; it never substitutes for real data.

7. Feature extraction

Mel (80 channels, 1024/256/1024 STFT, 22.05 kHz), frame-level F0 (pyworld `dio` + `stonemask`, hop-aligned), frame-level energy (STFT magnitude sum), then phoneme-level averaging per the reference. Pitch and energy are standardised with per-corpus statistics (`stats.json`); outliers are clipped using the reference's IQR rule only for scaler fitting. The preprocessor validates lengths, rejects invalid values and logs exclusions (`preprocess_report.json`).

8. Training configuration

Both languages use the reference hyper-parameters: 4-layer encoder / 6-layer decoder (256 hidden), batch 32, Adam with `betas (0.9, 0.98)`, warm-up 4 000 steps, duration L1 + pitch/energy L1 + mel MSE + post-net MSE. Training runs on a Colab T4 GPU with `English/train.py` / `Telugu/train.py`; checkpoints (`{step}.pth.tar`, `latest.pth.tar`, `final.pth.tar`) store model + optimizer + step and live in the per-language `checkpoints/` directories (symlinked to Google Drive in the Colab notebook so runs survive session disconnects). `--resume` restores optimizer state and global step.

`[PLACEHOLDER — pending training run: actual step budget, wall-clock time, final losses.]`

9. Inference and test sentences

`English/inference.py` / `Telugu/inference.py` synthesise the five assignment sentences per language (E1, E2a, E2b, E3, E4 / T1, T2a, T2b, T3, T4 — verbatim from the assignment PDF, pinned by tests) and write:

- `Results/<lang>/test_generated_audios/<ID>.wav`
- `Results/<lang>/variance_adaptor_plots/{duration,pitch,energy}/<ID>.png`
- `Results/<lang>/variance_adaptor_plots/<ID>_variance.json` (raw predicted values backing the plots)

Inference refuses to run without a trained checkpoint (item L).

10. Results

10.1 Training and validation losses

`[PLACEHOLDER — pending training run]` Figures: `Results/<lang>/training_loss.png` and `Results/<lang>/validation_loss.png`. The analysis (convergence, overfitting, instability) will describe exactly what the curves show.

10.2 Per-sentence observations

`[PLACEHOLDER — pending training run]` One row per test sentence with pronunciation / duration / quality notes, filled from listening to the generated wavs only.

10.3 Variance-adaptor plots

`[PLACEHOLDER — pending training run]` Per-sentence discussion of the duration, pitch and energy plots, backed by the `<ID>_variance.json` values.

11. English vs Telugu — comparison

Design-level differences (observed in code and data, not speculation):

Aspect	**English (Svarah)**	**Telugu (IndicTTS)**
Text front-end	CMU dict + `g2p_en`, ARPAbet phones with stress	rule-based G2P, language-specific phones
Number/time text	cleaned to words by `english_cleaners`	custom Telugu normalisation (10:45, 12వ)
Speakers	117 in the corpus; per-utterance IDs only via `meta_speaker_stats.csv` (see item B)	2 speakers from the gender field
Accents/voices	19 states' accents, read + spontaneous	2 studio voices, read stories
Aligner ASR	wav2vec2-large-960h (English)	Indic wav2vec2 v2 (Telugu)
Intra-word duration split	phone-class weights (estimated)	character-weighted G2P shares (estimated)

Empirical comparison of losses and generated speech: `[PLACEHOLDER — pending training run]`.

12. Validation and testing

Suite	**Command**	**Result**
Unit + integration (93 tests)	`python -m pytest tests/ -q`	PASS (1 skip = real-data gate)
End-to-end smoke (synthetic corpus -> preprocess -> train 12 steps -> inference -> plots)	`python tools/smoke_test.py`	PASS
Real-data decode + small real CTC alignment per language	`RUN_REAL_TESTS=1 python -m pytest tests/test_real_data.py -v -s`	see `VALIDATION.md`
Environment verification	`python tools/check_env.py`	PASS

Synthetic and real-data results are kept separate (the real-data suite is skipped unless explicitly enabled). Detailed commands, versions and limits are in `VALIDATION.md`; the per-test coverage map is in `tests/README.md`.

13. Reproducibility

```
# environment (Colab: skip the torch line)
pip install -r requirements.txt
python tools/check_env.py
```

```
# data + features (per language)
python English/preprocess.py            # download -> raw -> align -> features
python Telugu/preprocess.py

# training (Colab GPU)
python English/train.py
python Telugu/train.py

# inference + plots
python English/inference.py
python Telugu/inference.py

# report + tests
python tools/make_report_pdf.py
python -m pytest tests/ -q
python tools/smoke_test.py
```

The Colab notebook `notebooks/FastSpeech2_Colab.ipynb` walks a beginner through every step (project upload, Drive persistence for both languages' logs, HF token via Colab Secrets, alignment pilot, training, inference, packaging). The alignment engine depends on `ctc-segmentation` and `transformers` (pinned ranges in `requirements.txt`); no code depends on deprecated torchaudio alignment APIs.

14. Limitations and disclosures

1. **Intra-word phoneme timing is estimated** (section 6). Word timing is measured; the phoneme split inside a word is a weighted heuristic. This matches common practice when no word-level aligner exists for Telugu, but the report and the data treat these values as estimates.
2. **Svarah speaker IDs** depend on `meta_speaker_stats.csv`; without it the English model is an honestly documented single-voice model, not a fabricated 117-speaker model.
3. **No training results are claimed** before the runs exist (section 10).
4. The smoke test validates mechanics on synthetic audio; it is not evidence of speech quality.
5. HiFi-GAN universal vocoder (22.05 kHz) is used as-is from the reference; vocoder fine-tuning is out of scope.

15. Deliverable structure

```
project/
├── English/   train.py  preprocess.py  inference.py  config.yaml  checkpoints/
├── Telugu/    train.py  preprocess.py  inference.py  config.yaml  checkpoints/
├── fs2_core/  model/  preprocessor/  text/  audio/  hifigan/  lexicon/  ...
├── data_prep/ common.py  download_svarah.py  download_indic_tts_telugu.py
├── tests/     unit + integration suites (see tests/README.md)
├── tools/     smoke_test.py  check_env.py  make_report_pdf.py  build_notebook.py
├── notebooks/ FastSpeech2_Colab.ipynb
├── Results/   English/  Telugu/  (loss plots, generated audio, variance plots)
├── Report/    Technical_Report.md  Technical_Report.pdf
├── README.md  requirements.txt  CHANGELOG.md  VALIDATION.md
```

Attribution: encoder/decoder/variance-adaptor/training skeleton from [ming024/FastSpeech2](https://github.com/ming024/FastSpeech2) (MIT); HiFi-GAN vocoder weights from the same repository's bundled generators; CTC segmentation engine from `readbeyond/ctc-segmentation` (Apache 2.0).